

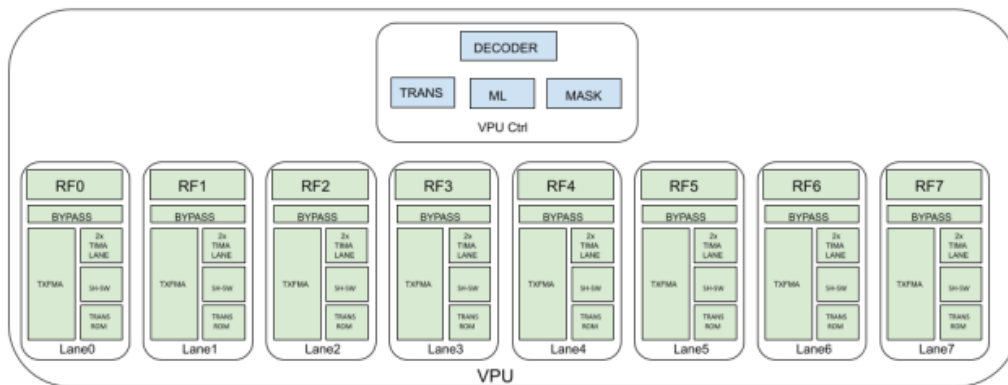
Architecture Proposal: Upgrading ET-SoC-1 VPU for RISC-V Vector Extension (v1.0) Compliance

1. VPU Overview

1.1 Micro-Architecture Overview

The VPU is composed of eight identical lanes and a control unit. Lanes always operate completely synchronously, so to the final programmer, the VPU appears as a single entity that performs eight operations per cycle. Each VPU lane contains:

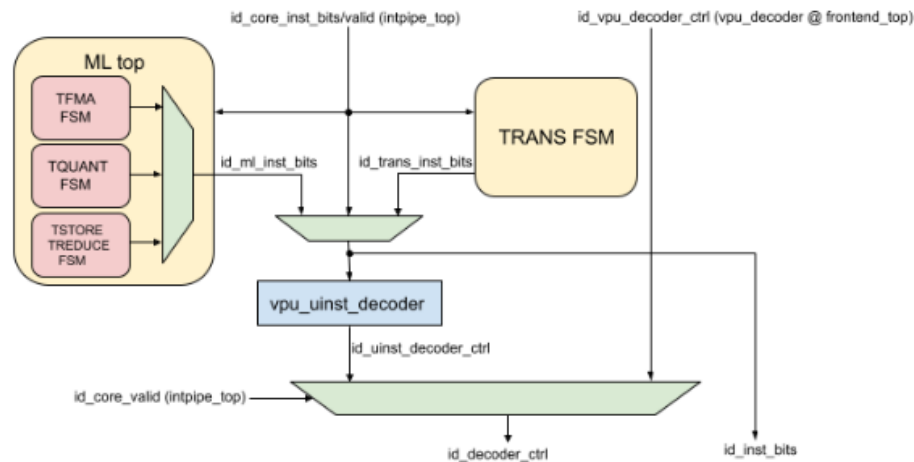
- A Register File (RF) holding sixty-four (32 x 2 threads) 32b-wide registers. When the programmer views them together, the eight lanes present a unified RF containing thirty-two 256b-wide registers.
- A Tensor (FP16/FP32) Fused Multiply-Add (TXFMA) unit capable of performing one 32b or two 16b Floating Point (FP) multiply-add operations
- Two Tensor INT8 Multiply-Add (TIMA) units capable of performing 8b integer multiply-accumulate operations
- An Integer (int) unit capable of performing 32b integer and logical operations
- A transcendental (trans) unit capable of performing 32b FP trans instructions (exp2, log2, reciprocal)



1.2 VPU Control Unit

The VPU control unit acts as the centralized component for controlling the whole VPU. It consists of the three major components:

1. Machine Learning (ML) unit
2. Transcendental FSM
3. u-instruction Decoder



Some Esperanto custom instructions require a sequence of simpler internal operations to implement complex functionality. The ML Unit generates the required u-instructions for tensor operations such as Tensor FMA (TFMA), TQuant, and TStore/TReduce, while the Transcendental FSM generates the sequence of u-instructions required for complex transcendental operations such as exponential and logarithmic functions. The generated u-instructions are subsequently decoded by the u-Instruction Decoder to produce the control signals required to operate the VPU lane functional units. The VPU can therefore receive execution instructions through either the u-Instruction Decoder, for sequenced or complex operations, or the VPU decoder instantiated in the Front-End, for instructions that can be issued directly to the VPU execution lanes.

1.3 Existing Instruction Classes

The VPU currently supports two categories of instructions

1. RISC-V scalar single-precision floating-point instructions
2. Esperanto custom instructions

The Esperanto custom instructions are further classified as

- a. Esperanto Packed-Single Instructions
- b. Esperanto Packed-Integer Instructions

c. Esperanto Tensor Instructions

1.3.1 RISC-V Scalar Floating-Point Instructions

The existing RISC-V floating-point instructions operate on 32-bit scalar operands.

For each instruction:

1. Two 32-bit operands are read.
2. The required floating-point operation is performed.
3. A 32-bit result is written to the destination register.

Although the existing register file spans 8 lanes and is viewed as 256 bits wide, these instructions require only 32 bits. Therefore, the current implementation uses only Lane 0 to make it architecturally scalar and 32 bits wide.

1.3.2 Esperanto Custom Instructions

Esperanto Packed-Single and Packed-Integer instructions operate across all eight lanes. Each lane operates on its corresponding lane, thereby producing a 256-bits wide register result. While the Tensor instructions have their own dedicated buffers to load tensors from L1 Scratchpad or Memory, their results are eventually written back into the existing 256-bit wide register.

2. Objective: RV-Vector v1.0 Compliance

The primary goal of this architectural enhancement is to extend the existing VPU to support RISC-V Vector Extension v1.0 while maintaining compatibility with existing Esperanto custom instructions. The initial implementation will target:

- Vector Floating-Point Instructions
- Support for SEW configuration 16 and 32
- LMUL = 1, initially
- Extension to LMUL > 1 through the proposed vector micro-instruction sequencer

The current lane architecture does not support INT16/INT32 operations, so initially, only Floating-Point vector instructions will be targeted for support.

3. Required Architectural Enhancements

The proposed implementation consists of four primary hardware enhancements:

1. Dedicated Floating-Point Register File (FPRF)
2. Dedicated Scoreboard for FPRF
3. RISC-V Vector CSRs and Vector State Management
4. Vector Micro-Instruction Sequencer

3.1 Dedicated Floating-Point Register File

The RISC-V architecture distinctly separates the following types of registers:

- Integer registers (x0 -- x31)
- Floating-point registers (f0 -- f31)
- Vector registers (v0 -- v31)

In the current implementation of ET-SoC-1, the RISC-V scalar floating-point instructions utilize the lower 32 bits of each 256-bit register as their architectural register. To ensure a clear architectural separation between Vector and scalar Floating-Point operations, a new dedicated Floating-Point Register File (FPRF) is proposed. This FPRF will contain a total of 64 registers, with 32 registers allocated for each thread, and each register is 32 bits wide.

The existing register file will continue to be used for:

1. RISC-V Vector registers, VLEN = 256 bits
2. Esperanto Packed-Singled instructions
3. Esperanto Packed-Integer instructions
4. Esperanto Tensor instructions

3.2 Dedicated Scoreboard

The introduction of a dedicated Floating-Point register file requires corresponding dependency tracking. Therefore, a dedicated scoreboard should be implemented for the new RFRF. The dependencies tracked are consumed by both the VPU and Integer pipeline.

3.3 RISC-V Vector CSRs and Architectural State

Beyond the vector register file and functional units, RISC-V Vector also implements its own Vector Control and Status registers for the configuration of vector operations. These registers include:

- **VSTART**: Used after returning from an exception
- **VXSAT**: Contains the saturation flag
- **VXRM**: Contains the rounding mode
- **VCSR**: Vector control and status register
- **VL**: Vector length to compute
- **VTYPER**: Vector type (SEW, LMUL, etc)
- **VLENB**: Vector length in bytes

To update the read-only CSRs, instructions such as `vset(i)` and `vl(i)` must be implemented. Additionally, necessary privilege mechanisms should also be incorporated to handle scenarios such as context-switching, exception handling, resuming after an exception, etc.

3.4 Vector Micro-Instruction Sequencer

The existing VPU architecture receives instructions from three primary sources.

1. Front-End
2. Machine-Learning Unit
3. Transcendental FSM

Except for the Front-End, the other two units can generate multiple micro-operations from a single architectural instruction, which are then decoded through the existing instruction decoder before being dispatched to the appropriate functional unit.

RISC-V Vector supports vector register grouping through the **LMUL** configuration.

For **LMUL** > 1, a logical vector register group spans multiple physical vector registers.

For example:

- a. **LMUL** = 1, `v0` is a single 256-bit register
- b. **LMUL** = 2, `v0` contains two consecutive 256-bit registers (`v0`, `v1`)

The existing VPU datapath naturally operates on one 256-bit register per cycle. Therefore, an **LMUL** > 1 instruction can be implemented by executing the corresponding operation over multiple consecutive physical registers.

Therefore, we propose a dedicated **Vector Micro-Instruction Sequencer** that, based on the **LMUL** configuration, issues multiple similar instructions to consecutive physical registers for the **LMUL** group. It will also manage lane masking based on elements stored in the "vl" CSR.

4. Future Optimization

4.1 Low Precision Tensor Support (FP8 and FP4)

AI chips are moving toward smaller data types such as FP8 and FP4 to perform more calculations with less memory and lower data movement. **NVIDIA introduced the Blackwell architecture in March 2024**, with support for FP4 and FP8 AI operations. **AMD introduced the Instinct MI350 Series in June 2025**, with support for low-precision formats including FP4 and FP8. Following this trend, ET-SoC-1 could extend its existing TXFMA units to support FP8 and FP4 Tensor FMA/MAC operations, providing a path toward higher AI inference performance and efficiency.